

Diseño y desarrollo de aplicaciones web

Breve descripción:

El componente formativo aborda los principios del diseño y desarrollo de aplicaciones web, incluyendo la organización de la información, el diseño de contenidos e interfaces, la maquetación mediante HTML y CSS, y los fundamentos de las tecnologías frontend. Estos elementos permiten comprender cómo se estructuran, presentan e implementan aplicaciones web funcionales, intuitivas y orientadas a la experiencia del usuario.

Tabla de contenido

Introducción	4
1. Arquitectura de la información en aplicaciones web	5
1.1. Organización de la información	6
1.2. Sistemas de navegación en sitios web	9
2. Diseño de contenido e información web	13
2.1. Características y distribución del contenido web	13
2.2. Formatos y tipos de archivos para la publicación de recursos.....	16
2.3. Diseño de información: wireframe, jerarquía de información y retículas	22
3. Diseño de interfaces web	26
3.1. Tipos de interfaces web	27
3.2. Diseño intuitivo y usable de interfaces.....	32
3.3. Mockup y prototipado de interfaces	34
3.4. Plataformas y herramientas para el diseño de interfaces	38
4. Maquetación de aplicaciones web	41
4.1. Lenguaje HTML y uso de etiquetas.....	41
4.2. Hojas de estilo CSS.....	50
4.3. Estilos y guías de estilo para la presentación web	61
5. Tecnologías frontend para el desarrollo web	67

5.1. Funcionalidades de la tecnología frontend	69
5.2. Tipos de datos y funciones en el desarrollo frontend	73
5.3. Uso de librerías en el desarrollo frontend	76
Síntesis	82
Glosario	84
Referencias bibliográficas	86
Créditos	87

Introducción

La creación de aplicaciones web modernas requiere integrar principios de organización de la información, diseño visual y desarrollo tecnológico. Estos elementos permiten estructurar contenidos de manera clara, facilitar la navegación de los usuarios y construir interfaces funcionales que respondan a diferentes necesidades digitales.

En el desarrollo de aplicaciones web, el diseño de la información cumple un papel fundamental, ya que permite definir cómo se organizan los contenidos, cómo se presentan y de qué forma interactúan los usuarios con el sistema. Para ello se emplean herramientas como wireframes, retículas y mockups, que ayudan a planificar la estructura visual y la experiencia de uso antes de iniciar la construcción de la aplicación.

De igual manera, la maquetación web mediante tecnologías como HTML y CSS permite transformar los diseños en interfaces funcionales accesibles desde diferentes dispositivos. Estas tecnologías hacen posible estructurar los contenidos, aplicar estilos visuales y mantener coherencia gráfica a través de guías de estilo.

Finalmente, el uso de tecnologías frontend amplía las capacidades de las aplicaciones web al permitir la incorporación de funciones dinámicas, el consumo de servicios y el uso de librerías que facilitan el desarrollo. De esta forma, el diseño y desarrollo de aplicaciones web integra aspectos conceptuales, visuales y tecnológicos que contribuyen a la creación de experiencias digitales eficientes y centradas en el usuario.

1. Arquitectura de la información en aplicaciones web

La arquitectura de la información en aplicaciones web se refiere a la forma en que se organizan, estructuran y presentan los contenidos dentro de un sitio o aplicación digital para facilitar su comprensión y uso por parte de los usuarios. Su propósito principal es permitir que la información sea fácil de encontrar, interpretar y utilizar, garantizando una navegación clara y una experiencia de usuario eficiente.

En el desarrollo de aplicaciones web, la arquitectura de la información permite definir cómo se distribuyen los contenidos, cómo se relacionan entre sí y de qué manera los usuarios pueden acceder a ellos. Una estructura adecuada facilita la interacción con la plataforma, mejora la usabilidad y contribuye a que los usuarios logren sus objetivos de forma rápida e intuitiva.

Para lograrlo, se analizan aspectos como la organización de la información, la jerarquía de contenidos y los sistemas de navegación. Estos elementos permiten estructurar la aplicación de manera lógica, asegurando que los usuarios comprendan dónde se encuentran dentro del sitio y cómo pueden desplazarse hacia otras secciones.

La arquitectura de la información también se relaciona con el diseño centrado en el usuario, ya que busca adaptar la estructura del contenido a las necesidades, expectativas y comportamientos de quienes utilizan la aplicación. Por esta razón, antes de diseñar interfaces o desarrollar funcionalidades, es fundamental definir cómo se organizará la información dentro del sistema.

En el contexto del desarrollo web, una arquitectura de la información bien definida contribuye a mejorar la usabilidad, optimizar la experiencia del usuario y facilitar el mantenimiento y crecimiento de las aplicaciones digitales. A partir de esta

base se construyen los sistemas de navegación, las interfaces y los elementos visuales que permiten la interacción con la aplicación.

1.1. Organización de la información

En el desarrollo de aplicaciones web, la organización de la información constituye un elemento fundamental para garantizar una experiencia de usuario eficiente, intuitiva y satisfactoria. A medida que aumenta la cantidad de contenidos y funcionalidades en una aplicación, se hace necesario estructurar la información de forma lógica, clara y accesible.

Una adecuada organización no solo facilita la navegación de los usuarios, sino que también contribuye a mejorar la usabilidad del sistema, optimizar el acceso a los contenidos y apoyar el cumplimiento de los objetivos de la aplicación.

La organización de la información forma parte de la arquitectura de la información, disciplina que se encarga de definir cómo se estructuran, relacionan y presentan los contenidos dentro de un entorno digital, como sitios web, aplicaciones móviles o plataformas virtuales. Esta fase se desarrolla después de identificar las necesidades del sistema y los requerimientos del usuario, permitiendo establecer una estructura coherente para la distribución del contenido.

La organización de la información corresponde al proceso de clasificar, estructurar y presentar los contenidos de una aplicación web de manera clara y comprensible, con el fin de que los usuarios puedan localizar fácilmente la información que necesitan.

Para lograrlo, la arquitectura de la información se apoya en varios elementos fundamentales:

- **Estructura:** define la forma en que se organizan los contenidos dentro de la aplicación. Puede adoptar distintos modelos, como estructuras jerárquicas, secuenciales o en red, dependiendo de la naturaleza del sistema y de las necesidades del usuario.
- **Navegación:** corresponde a los mecanismos que permiten al usuario desplazarse dentro de la aplicación, mediante menús, enlaces, botones o rutas de navegación que facilitan el acceso a las diferentes secciones.
- **Etiquetado:** consiste en asignar nombres claros y comprensibles a las categorías, secciones o funcionalidades de la aplicación, de manera que los usuarios puedan interpretar fácilmente la información disponible.
- **Búsqueda:** permite localizar contenidos específicos dentro de la aplicación a través de herramientas de búsqueda internas, lo que facilita el acceso rápido a la información cuando el volumen de contenidos es amplio.

Una adecuada organización de la información aporta múltiples beneficios en el diseño de aplicaciones web, entre los que se destacan:

- Mejora la experiencia del usuario.
- Facilita la navegación y el acceso a los contenidos.
- Reduce el tiempo de búsqueda de la información.
- Incrementa la eficiencia en el uso del sistema.
- Favorece el posicionamiento en motores de búsqueda (SEO).

Por ejemplo, en una aplicación de comercio electrónico, la organización de la información permite guiar al usuario durante el proceso de navegación y compra.

Figura 1. Estructura de un comercio electrónico.



En este tipo de aplicaciones suele utilizarse una estructura jerárquica, donde la página principal o inicio actúa como punto de acceso a las diferentes secciones del sistema.

Entre las secciones más comunes se encuentran:

- Categorías, que agrupan los productos en diferentes tipos como tecnología, ropa u hogar, facilitando la búsqueda de artículos.
- Ofertas, destinada a mostrar promociones o descuentos especiales.
- Carrito de compras, donde el usuario puede visualizar y gestionar los productos seleccionados antes de finalizar la compra.
- Perfil de usuario, que permite acceder a la información personal, historial de pedidos y configuraciones de la cuenta.

- Atención al cliente, que ofrece soporte, contacto y ayuda para resolver inquietudes o problemas.

Una estructura bien organizada permite que la aplicación presente una navegación clara, acceso rápido a la información y una mejor experiencia para el usuario, facilitando la interacción con el sistema.

1.2. Sistemas de navegación en sitios web

En el desarrollo de aplicaciones y sitios web, los sistemas de navegación constituyen un elemento fundamental para facilitar la interacción entre el usuario y el contenido. A medida que un sitio web incorpora mayor cantidad de información y funcionalidades, resulta necesario implementar mecanismos que permitan a los usuarios orientarse dentro del sistema y acceder a los contenidos de forma rápida y clara.

Los sistemas de navegación se definen como el conjunto de estructuras, elementos y mecanismos que permiten a los usuarios desplazarse dentro de un sitio o aplicación web. Su objetivo principal es guiar al usuario a través del contenido disponible, facilitando la localización de la información y mejorando la experiencia de uso.

Una navegación adecuada contribuye a:

- Facilitar el acceso a la información.
- Mejorar la experiencia de usuario (UX).
- Reducir el tiempo de búsqueda.

- Incrementar la permanencia en el sitio.
- Favorecer el posicionamiento en motores de búsqueda (SEO).

Los sitios web pueden utilizar diferentes tipos de navegación dependiendo de la estructura del contenido y de los objetivos de la aplicación.

a. Navegación global: corresponde al conjunto de enlaces principales que permiten acceder a las secciones más importantes del sitio web.

Generalmente se ubica en la parte superior de la página en forma de menú principal y se mantiene visible en todas las páginas del sitio.

Figura 2. Características de la navegación global.



b. Navegación local: permite al usuario desplazarse dentro de una sección específica del sitio web. Se utiliza para acceder a contenidos relacionados

que pertenecen a una misma categoría o área temática. Suele presentarse mediante menús secundarios, barras laterales o submenús.

- c. **Navegación contextual:** proporciona enlaces dentro del contenido de una página que permiten acceder a información relacionada con el tema que el usuario está consultando. Este tipo de navegación facilita la exploración de contenidos relacionados sin necesidad de regresar al menú principal. Por ejemplo, un artículo que incluye enlaces a contenidos similares o recomendaciones.
- d. **Navegación secuencial:** guía al usuario a través de una serie de pasos ordenados que deben seguirse de manera progresiva para completar una tarea o proceso específico. Por ejemplo:
 - Paso 1. Registro.
 - Paso 2. Confirmación.
 - Paso 3. Finalización.
- e. **Navegación por breadcrumbs:** también conocida como migas de pan, muestra la ubicación del usuario dentro de la estructura jerárquica del sitio web, permitiendo regresar fácilmente a niveles superiores. Por ejemplo:

Inicio > Productos > Electrónica > Celulares

Los sistemas de navegación están conformados por diversos elementos que facilitan el acceso a la información dentro de un sitio web:

- Menús principales, secundarios o desplegables.
- Enlaces internos y externos.
- Botones de acción.
- Iconos de apoyo visual.

- Barra de búsqueda.

Para diseñar un sistema de navegación efectivo se recomienda:

- Utilizar estructuras simples y claras.
- Emplear etiquetas comprensibles.
- Limitar el número de opciones en los menús.
- Mantener coherencia en todas las páginas.
- Incluir herramientas de búsqueda.
- Diseñar navegación adaptable a diferentes dispositivos.
- Priorizar la experiencia del usuario.

Un sistema de navegación mal diseñado puede generar desorientación, dificultar la localización de la información y provocar que los usuarios abandonen el sitio. Por esta razón, la planificación de la navegación constituye un aspecto fundamental en el diseño de aplicaciones web.

2. Diseño de contenido e información web

El diseño de contenido e información web es un proceso que se enfoca en estructurar, organizar y presentar los contenidos digitales de manera clara, accesible y comprensible para los usuarios. Este proceso integra aspectos de comunicación, diseño visual y experiencia de usuario (UX), con el propósito de facilitar la interacción con los sitios o aplicaciones web.

Un diseño adecuado del contenido permite que la información se distribuya de forma lógica, facilitando la lectura, la navegación y el acceso a los recursos disponibles. Para lograrlo, se aplican principios relacionados con la organización del contenido, el uso de formatos digitales apropiados y el diseño de estructuras visuales que orienten al usuario dentro del entorno web.

En este contexto, el diseño de información se apoya en herramientas y técnicas que permiten planificar la estructura de las páginas, establecer jerarquías de contenido y definir la disposición de los elementos en la interfaz. De esta manera, se busca mejorar la comprensión del contenido, optimizar la experiencia del usuario y garantizar que la información cumpla su propósito dentro del sistema digital.

2.1. Características y distribución del contenido web

El contenido web constituye uno de los elementos más importantes dentro del diseño de aplicaciones digitales, ya que representa la información que los usuarios consultan, interpretan e interactúan dentro de un sitio web. Una adecuada organización del contenido permite que la información sea clara, accesible y fácil de

comprender, lo que mejora la experiencia del usuario y facilita la navegación dentro del sistema.

En el diseño de contenido web es necesario considerar una serie de características que orientan la forma en que se presentan los elementos visuales, los textos y los recursos interactivos dentro de una página.

Entre las principales características del diseño de contenido web se encuentran las siguientes:

- **Estética:** corresponde a la apariencia visual del sitio web. El uso adecuado del color, la tipografía y la composición visual contribuye a generar una experiencia agradable y a fortalecer la identidad visual del sitio.
- **Usabilidad:** hace referencia a la facilidad con la que los usuarios pueden interactuar con una página web. Un diseño usable permite que las personas encuentren la información rápidamente y realicen sus tareas sin dificultades.
- **Interactividad:** se relaciona con la capacidad del sistema para responder a las acciones del usuario. Elementos como botones, formularios, menús o animaciones permiten que la interacción sea dinámica y participativa.
- **Estética:** corresponde a la apariencia visual del sitio web. El uso adecuado del color, la tipografía y la composición visual contribuye a generar una experiencia agradable y a fortalecer la identidad visual del sitio.
- **Responsividad:** permite que el contenido web se adapte automáticamente a diferentes dispositivos y tamaños de pantalla, como computadores, tabletas y teléfonos móviles.

- **Consistencia:** garantiza que los elementos de diseño mantengan un mismo estilo visual y funcional en todas las páginas del sitio, facilitando el aprendizaje y la navegación del usuario.
- **Accesibilidad:** busca que el contenido pueda ser utilizado por todas las personas, incluyendo aquellas con limitaciones visuales, auditivas o motoras, mediante el uso de buenas prácticas de diseño inclusivo.

Además de estas características, es importante considerar la forma en que el contenido se distribuye dentro de la página. La distribución del contenido web determina cómo se organiza la información, permitiendo que los usuarios identifiquen fácilmente los elementos principales y naveguen de manera eficiente.

Tabla 1. Elementos de la distribución del contenido web.

Elemento	Descripción	Aspectos clave	Ejemplo
Estructura del contenido.	Organización jerárquica de la información.	Títulos, subtítulos y orden lógico.	Artículo dividido en secciones.
Jerarquía visual.	Destaca los elementos más importantes.	Tamaño, color y posición.	Botón destacado.
Espacio en blanco.	Espacios que mejoran la lectura.	Márgenes y separación.	Texto con espacios amplios.
Diseño en rejilla (Grid).	Organización en filas y columnas.	Alineación y consistencia.	Página con columnas.
Navegación.	Forma de desplazarse dentro del sitio.	Menús y enlaces claros.	Menú superior.
Responsive (Adaptabilidad).	Adaptación a diferentes dispositivos.	Diseño flexible.	Menú hamburguesa en celular.

Elemento	Descripción	Aspectos clave	Ejemplo
Tipografía y legibilidad.	Forma en que se presenta el texto.	Tamaño, fuente y contraste.	Fuente clara.
Contenido visual.	Uso de imágenes y multimedia.	Calidad y apoyo visual.	Infografías.
Escaneo de contenido.	Facilita la lectura rápida.	Listas y párrafos cortos.	Uso de viñetas.
Llamados a la acción (CTA).	Elementos que invitan a actuar.	Botones visibles.	Comprar ahora.

Una correcta distribución del contenido facilita la lectura, mejora la comprensión de la información y permite que los usuarios interactúen con el sitio web de manera más eficiente.

2.2. Formatos y tipos de archivos para la publicación de recursos

La publicación de recursos web constituye un proceso fundamental en el desarrollo y gestión de sitios y aplicaciones digitales. Este proceso implica el uso de diversos formatos y tipos de archivos que permiten estructurar el contenido, definir la apariencia visual, incorporar elementos multimedia y agregar funcionalidades interactivas.

Cada tipo de archivo cumple una función específica dentro del funcionamiento de una página web. La correcta selección y uso de estos formatos influye directamente en el rendimiento del sitio, la experiencia del usuario, la accesibilidad, la seguridad y el posicionamiento en los motores de búsqueda (SEO). Por esta razón, los desarrolladores

web deben conocer las características y aplicaciones de los principales formatos utilizados en la publicación de recursos digitales.

Un recurso web es cualquier archivo digital que puede ser consultado a través de Internet mediante un navegador web y que se encuentra almacenado en un servidor. Estos recursos se transmiten generalmente utilizando protocolos como HTTP o HTTPS.

Los recursos web pueden clasificarse de acuerdo con su función dentro del sitio o aplicación, entre los que se destacan los archivos de estructura, de presentación, de programación y los archivos multimedia.

a. Archivos de estructura y contenido: son aquellos que definen la organización básica del contenido dentro de una página web. El lenguaje principal utilizado para este propósito es HTML (HyperText Markup Language), cuyos archivos se identifican generalmente con las extensiones .html o .htm, utilizadas para representar documentos web interpretados por los navegadores.

HTML permite organizar la información mediante etiquetas que estructuran elementos como títulos, párrafos, imágenes, enlaces, listas, tablas y formularios. Gracias a esta estructura, los navegadores pueden interpretar y presentar correctamente el contenido de una página web.

Entre sus principales características se destacan:

- Está basado en etiquetas o tags.
- Permite integrar diferentes tipos de contenido.
- Constituye la base de cualquier sitio web.

Entre sus ventajas se encuentran:

- Es un estándar universal utilizado en toda la web.
- Es compatible con todos los navegadores.
- Favorece la accesibilidad y el posicionamiento en buscadores.

b. Archivos de presentación y diseño: los archivos de presentación se encargan de definir la apariencia visual del contenido web. Para ello se utiliza CSS (Cascading Style Sheets), cuyos archivos se guardan con la extensión .css, la cual permite aplicar estilos y formatos a los documentos HTML.

CSS permite controlar aspectos visuales como colores, tipografías, tamaños, márgenes, alineaciones y disposición de los elementos en la página. Su principal función es separar el contenido de su presentación, lo que facilita la organización del código y mejora el mantenimiento del sitio.

Entre sus principales ventajas se encuentran:

- Permite separar el contenido del diseño visual.
- Facilita la reutilización de estilos en diferentes páginas.
- Mejora la consistencia visual y la experiencia del usuario.

c. Archivos de programación y comportamiento: permiten añadir funcionalidad e interactividad a las páginas web. Para ello se utiliza JavaScript, cuyos archivos se guardan con la extensión .js, empleada para

implementar scripts que controlan el comportamiento dinámico de las páginas web.

JavaScript se ejecuta principalmente en el navegador del usuario y permite realizar acciones dinámicas como validar formularios, actualizar contenido sin recargar la página, crear animaciones o comunicarse con servidores.

Entre sus características principales se destacan:

- Permite crear interfaces interactivas.
- Facilita la manipulación del contenido de la página.
- Permite la comunicación con servidores y servicios web.

Además de los archivos de estructura, diseño y programación, los sitios web utilizan diversos formatos multimedia que enriquecen la experiencia del usuario.

Tabla 2. Formatos de imagen utilizados en la web.

Formato	Extensión	Uso principal	Características
JPEG	.jpg	Fotografías.	Alta compresión y buena calidad.
PNG	.png	Imágenes con transparencia.	Mayor calidad y soporte de transparencia.
GIF	.gif	Animaciones simples.	Soporte limitado de colores.
SVG	.svg	Gráficos vectoriales.	Escalables sin pérdida de calidad.
WebP	.webp	Imágenes optimizadas.	Menor tamaño con buena calidad.



Tabla 3. Formatos de audio utilizados en la web.

Formato	Características	Uso
MP3	Buena compresión y compatibilidad.	Música y podcasts.
WAV	Alta calidad de sonido.	Producción profesional.
OGG	Formato abierto y libre.	Alternativa a MP3.

Tabla 4. Formatos de video utilizados en la web.

Formato	Características	Uso
MP4	Alta compatibilidad y compresión.	Estándar más utilizado en la web.
WebM	Formato abierto optimizado para web.	Reproducción en navegadores.
AVI	Alta calidad pero archivos pesados.	Poco recomendado para web.

Para garantizar un buen rendimiento y una adecuada experiencia de usuario, es importante aplicar buenas prácticas al momento de publicar recursos web. Entre las más recomendadas se encuentran:

- Comprimir imágenes y videos para reducir el tiempo de carga.
- Utilizar formatos modernos y optimizados como WebP.
- Minimizar archivos CSS y JavaScript.
- Implementar técnicas de carga diferida (lazy loading).
- Verificar la compatibilidad entre diferentes navegadores.
- Optimizar los recursos para mejorar el posicionamiento SEO.

En un entorno digital cada vez más exigente, conocer y aplicar correctamente los formatos y tipos de archivos utilizados en la web permite desarrollar sitios más rápidos, eficientes y accesibles, contribuyendo a mejorar la experiencia de los usuarios y el desempeño general del sistema.

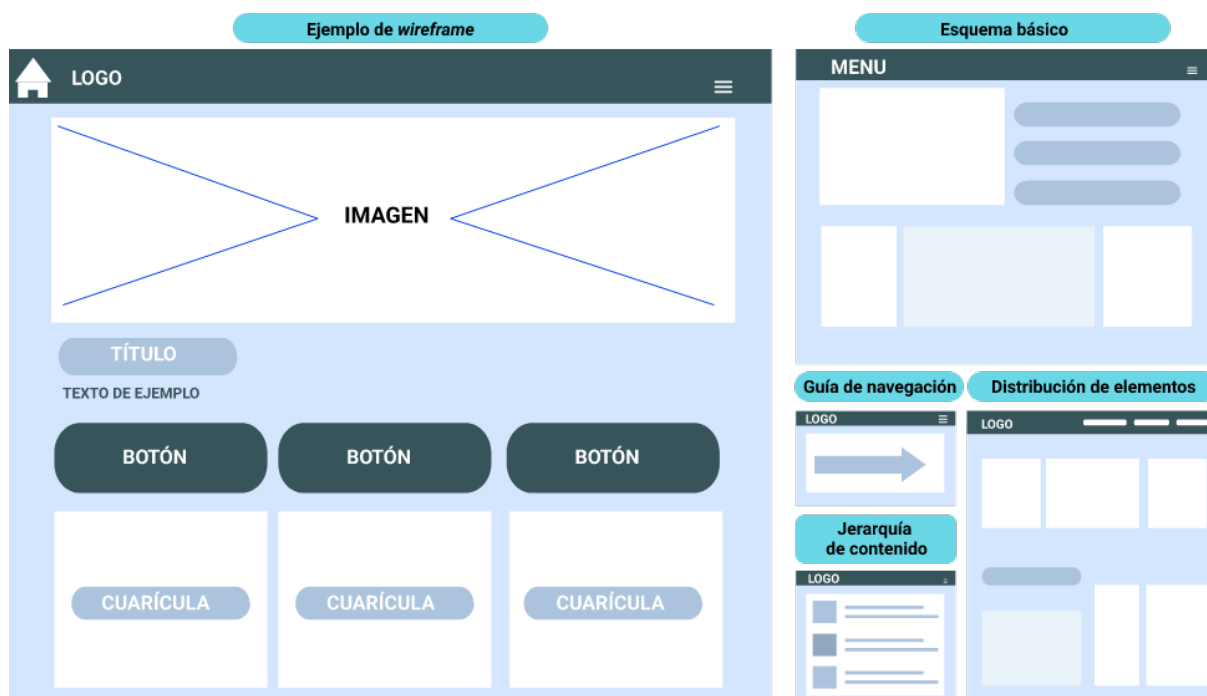
2.3. Diseño de información: wireframe, jerarquía de información y retículas

En el desarrollo de aplicaciones web y soluciones digitales, el diseño de la información permite organizar y presentar los contenidos de manera clara, lógica y accesible para los usuarios. No basta con que un sistema funcione correctamente desde el punto de vista técnico; también es necesario que la información esté estructurada de forma comprensible para facilitar la navegación, la interacción y el uso eficiente del sistema.

Para lograrlo, se utilizan diferentes herramientas de planificación que ayudan a definir cómo se distribuirá la información dentro de una interfaz digital. Entre las más utilizadas se encuentran el wireframe, la jerarquía de la información y las retículas, las cuales permiten transformar los requerimientos del sistema en estructuras visuales que orientan tanto el diseño como el desarrollo.

El wireframe es una representación visual básica de una interfaz digital. Se trata de un esquema o prototipo que presenta la distribución de los elementos dentro de una página o pantalla, como menús, encabezados, botones, imágenes y áreas de contenido. Su objetivo es definir la estructura general de la interfaz antes de aplicar detalles gráficos.

Figura 3. Características de wireframe.



Entre las principales características del wireframe se encuentran:

- Representar de forma esquemática la estructura de la interfaz.
- Enfocarse en la funcionalidad más que en la apariencia visual.
- Permitir realizar cambios de manera rápida durante el proceso de diseño.
- Facilitar la comunicación entre diseñadores, desarrolladores y otros miembros del equipo.

Para complementar la información, acceder al video ¿Cómo realizar un wireframe? <https://www.youtube.com/watch?v=94UTvXQgRfM>

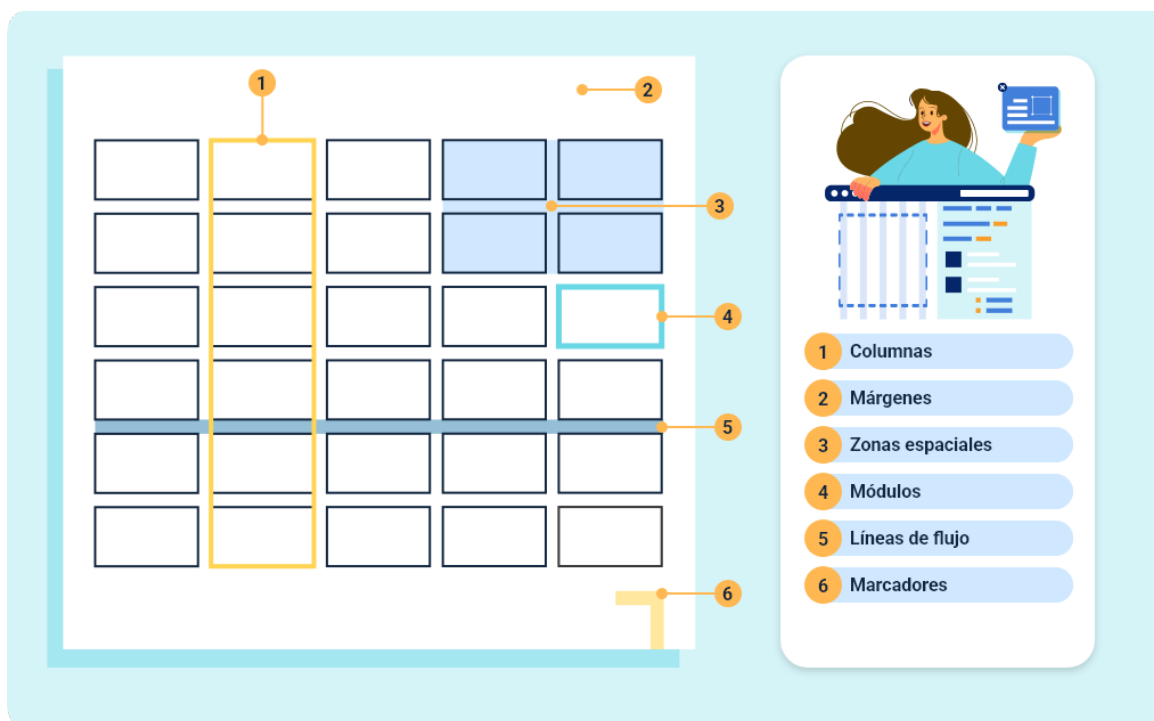
Otro elemento fundamental es la jerarquía de la información, que consiste en organizar los contenidos según su nivel de importancia. Una correcta jerarquía permite que el usuario identifique fácilmente los elementos principales y comprenda el orden en el que debe recorrer la información.

Para establecer la jerarquía de la información se utilizan diferentes recursos, como:

- Tamaño de títulos y textos.
- Uso del color para resaltar información relevante.
- Espaciado entre elementos.
- Posición de los contenidos dentro de la pantalla.
- Agrupación de información relacionada.

Por su parte, las retículas o grid systems son estructuras invisibles que permiten organizar el contenido dentro de una interfaz mediante la división del espacio en filas y columnas. Estas estructuras facilitan la alineación y distribución de los elementos, contribuyendo a crear diseños ordenados y coherentes.

Figura 4. Partes de una retícula.



Nota. Tomado de Inesem Business School (s. f.).

Entre los beneficios del uso de retículas se encuentran:

- Mejorar la organización visual del contenido.
- Aumentar la consistencia del diseño en todas las páginas del sitio.
- Facilitar el desarrollo de interfaces adaptables a diferentes dispositivos.
- Permitir una implementación más estructurada en el desarrollo web.

En conjunto, el wireframe, la jerarquía de la información y las retículas se complementan dentro del proceso de diseño. Mientras el wireframe define la estructura inicial de la interfaz, la jerarquía organiza la importancia del contenido y las retículas permiten distribuir los elementos de manera ordenada dentro del espacio disponible.

3. Diseño de interfaces web

El diseño de interfaces web es el proceso mediante el cual se planifica, organiza y construye la forma en que los usuarios interactúan con un sitio web o aplicación digital. Una interfaz web funciona como el punto de conexión entre el usuario y el sistema, ya que a través de ella se presentan los contenidos, las funciones y las herramientas que permiten navegar, consultar información y ejecutar diferentes acciones dentro de una plataforma digital.

En el desarrollo de soluciones tecnológicas, el diseño de interfaces no se limita únicamente al aspecto visual de una página. También implica la organización de la información, la disposición de los elementos interactivos, la claridad en la navegación y la facilidad con la que los usuarios pueden comprender y utilizar el sistema. Por esta razón, el diseño de interfaces integra principios de experiencia de usuario, usabilidad, accesibilidad y arquitectura de la información, con el propósito de crear entornos digitales funcionales y fáciles de utilizar.

Una interfaz bien diseñada permite que los usuarios encuentren rápidamente lo que buscan, comprendan cómo interactuar con el sistema y realicen sus tareas de manera eficiente. Esto contribuye a mejorar la satisfacción del usuario, optimizar el uso de la aplicación y fortalecer la calidad del producto digital.

En el contexto del desarrollo web, el diseño de interfaces también facilita el trabajo conjunto entre diseñadores y desarrolladores, ya que establece una guía clara para la construcción visual y funcional del sistema. De esta manera, se garantiza que los componentes gráficos, los elementos interactivos y la estructura del contenido respondan a los objetivos del proyecto y a las necesidades de los usuarios.

3.1. Tipos de interfaces web

Las interfaces web pueden clasificarse según su forma de funcionamiento, el nivel de interacción que ofrecen al usuario y la manera en que presentan la información dentro de un sitio o aplicación. Comprender estos tipos permite seleccionar el enfoque de diseño más adecuado según las necesidades del proyecto y el tipo de experiencia que se desea ofrecer al usuario.

A continuación, se presentan algunos de los tipos de interfaces web más utilizados en el desarrollo de sitios y aplicaciones digitales.

- a. Interfaces web estáticas:** presentan contenido fijo que no cambia en función de la interacción del usuario. En este tipo de interfaces, la información se visualiza de la misma manera para todos los visitantes, sin realizar modificaciones dinámicas en el contenido.

Este tipo de interfaz suele desarrollarse principalmente con HTML y CSS, lo que permite crear páginas sencillas, rápidas y fáciles de mantener.

Entre sus principales características se destacan:

- Contenido fijo que no depende de bases de datos.
- Estructura simple y de fácil implementación.
- Alta velocidad de carga.
- Bajo consumo de recursos del servidor.

Las interfaces estáticas se utilizan comúnmente en:

- Páginas institucionales informativas.
- Portafolios personales.

- Sitios web básicos o landing pages.

Cómo crear una interfaz web estática

De manera general, la creación de una interfaz web estática puede realizarse mediante los siguientes pasos:

- Obtener un servicio de alojamiento web (hosting) y registrar un dominio para el sitio.
- Diseñar la estructura de las páginas utilizando HTML.
- Aplicar estilos visuales mediante hojas de estilo CSS.
- Publicar los archivos del sitio en el servidor para que puedan ser accesibles desde Internet.

b. Interfaces web dinámicas: permiten que el contenido cambie de acuerdo con la interacción del usuario o con la información almacenada en bases de datos. En este tipo de interfaces, los usuarios pueden enviar información, realizar consultas y recibir respuestas personalizadas.

Para su funcionamiento se utilizan tecnologías como JavaScript, así como lenguajes de programación del lado del servidor y sistemas de gestión de bases de datos.

Entre sus principales características se encuentran:

- Contenido generado de manera dinámica.
- Interacción constante con el usuario.
- Integración con bases de datos.
- Actualización de información en tiempo real o bajo demanda.

Este tipo de interfaz se utiliza en:

- Plataformas educativas.
- Tiendas en línea.
- Sistemas de gestión empresarial.
- Redes sociales y aplicaciones web.

Cómo crear una interfaz web dinámica

De manera general, el desarrollo de una interfaz web dinámica puede realizarse mediante los siguientes pasos:

- Elegir la plataforma o tecnología adecuada para el desarrollo del sitio web.
- Diseñar y construir la estructura de la página utilizando HTML, CSS y JavaScript.
- Incorporar funcionalidades dinámicas mediante programación y conexión con bases de datos o servicios del servidor.
- Realizar pruebas de funcionamiento y publicar el sitio web en el servidor para su uso por parte de los usuarios.

c. Interfaces web responsivas: son aquellas que se adaptan automáticamente a diferentes tamaños de pantalla y dispositivos, como computadores, tabletas o teléfonos móviles. Este enfoque de diseño busca garantizar que el contenido sea accesible y funcional sin importar el dispositivo utilizado.

La implementación de interfaces responsivas se basa en el uso de técnicas de diseño flexible, hojas de estilo adaptativas y estructuras de diseño que

permiten reorganizar los elementos de la página según el tamaño de la pantalla.

Entre sus principales características se destacan:

- Adaptación automática a distintos dispositivos.
- Mejor experiencia de navegación en móviles.
- Uso de estructuras flexibles de diseño.
- Mayor accesibilidad y usabilidad.

En la actualidad, el diseño responsivo es un estándar en el desarrollo web, debido al crecimiento del uso de dispositivos móviles para acceder a Internet.

Cómo crear una interfaz web responsiva

De manera general, el desarrollo de una interfaz web responsiva puede realizarse mediante los siguientes pasos:

- Diseñar la estructura del sitio considerando diferentes tamaños de pantalla.
- Utilizar CSS con técnicas de diseño flexible, como Flexbox, CSS Grid y media queries.
- Optimizar imágenes, tipografías y elementos visuales para distintos dispositivos.
- Realizar pruebas en diferentes resoluciones y dispositivos para garantizar su correcta visualización.

d. Interfaces web orientadas a aplicaciones (Web Apps): ofrecen una experiencia similar a la de una aplicación instalada en un computador o

dispositivo móvil. Estas interfaces permiten realizar tareas complejas directamente desde el navegador, con altos niveles de interactividad.

Generalmente utilizan tecnologías modernas de desarrollo web que permiten actualizar el contenido sin necesidad de recargar completamente la página.

Entre sus principales características se encuentran:

- Alta interactividad con el usuario.
- Actualización dinámica del contenido.
- Integración con múltiples servicios y bases de datos.
- Experiencia de uso similar a una aplicación móvil o de escritorio.

Este tipo de interfaz se encuentra en plataformas como:

- Sistemas de gestión empresarial.
- Aplicaciones colaborativas en línea.
- Plataformas de productividad y almacenamiento en la nube.

Cómo crear una interfaz web orientada a aplicaciones

De manera general, el desarrollo de una interfaz web orientada a aplicaciones puede realizarse mediante los siguientes pasos:

- Definir las funcionalidades y tareas que realizará la aplicación.
- Diseñar la estructura de la interfaz y los componentes interactivos.
- Implementar la lógica del sistema utilizando tecnologías frontend y su integración con el backend.

- Realizar pruebas de funcionamiento, rendimiento y experiencia de usuario antes de su publicación.

3.2. Diseño intuitivo y usable de interfaces

En el desarrollo de interfaces web, no basta con crear entornos visualmente atractivos. Es fundamental que las interfaces sean fáciles de comprender, eficientes de utilizar y agradables para el usuario. En este sentido, el diseño intuitivo y la usabilidad se convierten en aspectos clave para garantizar una interacción efectiva entre las personas y los sistemas digitales.

El diseño intuitivo se refiere a la capacidad que tiene una interfaz para ser comprendida y utilizada de manera natural, sin que el usuario necesite instrucciones complejas o capacitación previa. Una interfaz intuitiva aprovecha patrones de interacción conocidos, de modo que el usuario puede identificar rápidamente cómo navegar, dónde hacer clic o cómo completar una tarea.

Este tipo de diseño se apoya en la experiencia previa de los usuarios con otros sistemas digitales, lo que permite reducir la confusión y facilitar el aprendizaje. Cuando una interfaz es intuitiva, los usuarios pueden concentrarse en sus objetivos y no en entender cómo funciona la herramienta.

Entre las principales características del diseño intuitivo se encuentran:

- Presentación simple y clara de los elementos de la interfaz
- Uso de patrones de interacción conocidos.
- Claridad en los mensajes, botones y acciones.
- Reducción de la carga cognitiva del usuario.

- Coherencia en la organización y funcionamiento de los elementos.

Por otra parte, la usabilidad se enfoca en evaluar qué tan fácil, eficiente y satisfactorio resulta para un usuario interactuar con un sistema. No se trata únicamente de cómo debería funcionar una interfaz en teoría, sino de cómo funciona realmente cuando las personas la utilizan.

La usabilidad busca que los usuarios puedan realizar sus tareas con el menor esfuerzo posible, reduciendo errores y mejorando la eficiencia en la interacción.

Entre los principales componentes de la usabilidad se encuentran:

- **Eficacia:** que se refiere a la capacidad del usuario para completar correctamente las tareas propuestas.
- **Eficiencia:** relacionada con el tiempo y el esfuerzo que el usuario necesita para lograr sus objetivos.
- **Satisfacción:** que corresponde a la percepción positiva del usuario durante la interacción con el sistema.

El diseño intuitivo y usable se aplica en numerosos contextos digitales. En plataformas educativas, por ejemplo, permite que los aprendices se concentren en el aprendizaje sin dificultades para navegar por los contenidos. En entornos empresariales, contribuye a mejorar la productividad y disminuir errores en los procesos.

Asimismo, en servicios digitales como la banca en línea, el comercio electrónico o los sistemas de salud, la usabilidad es especialmente importante, ya que una interfaz confusa puede generar errores, desconfianza o abandono del sistema.

Para lograr interfaces realmente intuitivas y usables, se suelen aplicar algunos principios fundamentales de diseño, entre los cuales destacan:

- Simplicidad, evitando elementos innecesarios que puedan generar distracciones.
- Consistencia, manteniendo patrones visuales y de interacción similares en todo el sistema.
- Retroalimentación, proporcionando respuestas claras ante las acciones del usuario.
- Visibilidad, mostrando de forma clara las opciones y funciones disponibles.
- Control del usuario, permitiendo que las personas tengan la posibilidad de corregir acciones o decidir cómo interactuar con el sistema.

En conclusión, el diseño intuitivo y usable de interfaces no solo mejora la experiencia del usuario, sino que también aumenta la eficiencia del sistema y facilita la adopción de las soluciones digitales. Por esta razón, constituye uno de los pilares fundamentales en el desarrollo de interfaces web modernas.

3.3. Mockup y prototipado de interfaces

En el proceso de diseño de interfaces web, es fundamental contar con herramientas que permitan visualizar, analizar y validar la estructura y el funcionamiento de una interfaz antes de su desarrollo final. En este contexto, los mockups y los prototipos se utilizan como representaciones visuales que facilitan la planificación, la comunicación entre equipos y la evaluación temprana de las soluciones de diseño.

El uso de estas herramientas permite detectar posibles problemas de usabilidad, organización de contenidos o interacción antes de iniciar la etapa de programación, lo que contribuye a optimizar el proceso de desarrollo y reducir errores en fases posteriores.

a. Mockup en el diseño de interfaces: un mockup es una representación visual detallada de una interfaz que muestra cómo se verá el diseño final de una página web o aplicación. A diferencia del wireframe, que se centra principalmente en la estructura, el mockup incluye elementos visuales más definidos como colores, tipografías, iconos, imágenes y estilos gráficos.

El objetivo del mockup es ofrecer una vista cercana al diseño final del producto digital, permitiendo evaluar la apariencia estética y la organización visual de la interfaz.

Entre las principales características de un mockup se encuentran:

- Representación visual de alta fidelidad.
- Inclusión de elementos gráficos y estilos visuales.
- Definición de tipografías, colores y componentes de diseño.
- Presentación cercana al diseño final del producto.

El mockup es especialmente útil para validar decisiones de diseño con clientes, diseñadores y desarrolladores, ya que permite visualizar de forma clara cómo se estructurarán los elementos en la interfaz.

Cómo crear un mockup de una interfaz

- **Definir los objetivos de la interfaz:** se identifican las necesidades del usuario y las funcionalidades que debe cumplir la página o aplicación.
- **Tomar como base el wireframe:** El mockup generalmente se construye a partir de un wireframe previamente elaborado, en el cual ya se ha definido la estructura de la interfaz.
- **Incorporar elementos visuales:** se agregan colores, tipografías, iconos, botones, imágenes y estilos gráficos que representen el diseño final.
- **Organizar los componentes de la interfaz:** se ajusta la distribución de menús, secciones, formularios y contenidos para asegurar una presentación clara y coherente.
- **Revisar y validar el diseño:** finalmente, el mockup se revisa con el equipo de diseño o desarrollo para realizar ajustes antes de avanzar hacia el prototipado o la programación.

b. Prototipado de interfaces: consiste en la creación de una versión simulada o interactiva de una interfaz que permite representar el funcionamiento de un sistema antes de su implementación definitiva. A diferencia del mockup, el prototipo puede incluir navegación entre pantallas, botones interactivos y simulaciones de comportamiento del sistema.

El objetivo del prototipo es evaluar la interacción del usuario con la interfaz, identificar posibles mejoras y validar la experiencia de uso antes de iniciar la etapa de desarrollo.

Entre las características principales del prototipado se destacan:

- Simulación de la navegación entre diferentes pantallas.
- Interacción con botones, menús y elementos funcionales.
- Evaluación temprana de la experiencia de usuario.
- Posibilidad de realizar pruebas con usuarios.

Los prototipos pueden desarrollarse con distintos niveles de detalle, dependiendo del momento del proceso de diseño.

- Prototipos de baja fidelidad, que representan ideas iniciales mediante esquemas simples.
- Prototipos de media fidelidad, que incluyen mayor organización de contenidos y algunos elementos visuales.
- Prototipos de alta fidelidad, que simulan con gran precisión el funcionamiento y la apariencia final del sistema.

Cómo crear un prototipo de interfaz

- **Definir los objetivos y funcionalidades:** se identifican las tareas que el usuario podrá realizar dentro de la interfaz y los elementos interactivos que serán necesarios.
- **Tomar como base el mockup o el wireframe:** el prototipo se construye a partir del diseño visual previamente definido, incorporando los elementos que simulan la interacción.
- **Agregar interacciones y navegación:** se establecen enlaces entre pantallas, botones funcionales, menús desplegables u otros componentes que permitan simular el comportamiento real de la interfaz.

- **Realizar pruebas con usuarios:** el prototipo se utiliza para evaluar la experiencia de uso, identificando dificultades de navegación, comprensión o interacción.
- **Ajustar y mejorar el diseño:** con base en los resultados de las pruebas, se realizan modificaciones antes de iniciar el desarrollo definitivo del sistema

El uso de mockups y prototipos aporta múltiples beneficios dentro del desarrollo de interfaces web:

- Facilita la visualización de la interfaz antes de programarla.
- Permite validar decisiones de diseño con diferentes actores del proyecto.
- Mejora la comunicación entre diseñadores, desarrolladores y clientes.
- Reduce errores en etapas posteriores del desarrollo.
- Favorece la evaluación de la experiencia de usuario.

En conjunto, estas herramientas permiten transformar ideas y requerimientos en representaciones visuales claras que guían el proceso de diseño y desarrollo de interfaces digitales.

3.4. Plataformas y herramientas para el diseño de interfaces

El diseño de interfaces web requiere el uso de plataformas y herramientas especializadas que permiten planificar, visualizar y probar la estructura y apariencia de una aplicación antes de su desarrollo final. Estas herramientas facilitan la creación de wireframes, mockups y prototipos interactivos, lo que contribuye a mejorar la

comunicación entre diseñadores, desarrolladores y otros integrantes del equipo de trabajo.

Actualmente existen diversas plataformas que apoyan el proceso de diseño de interfaces, cada una con características particulares orientadas a la colaboración, la creación visual o la simulación de interacciones. A continuación, se presentan algunas de las herramientas más utilizadas en el diseño de interfaces digitales.

Tabla 5. Herramientas más utilizadas.

Herramienta	Tipo de herramienta	Características principales	Uso en el diseño de interfaces
Figma	Plataforma de diseño colaborativo en línea.	Permite diseñar interfaces, crear prototipos interactivos y trabajar de forma colaborativa en tiempo real.	Diseño de interfaces web, creación de mockups y prototipos.
Adobe XD	Herramienta de diseño de experiencia de usuario.	Facilita la creación de wireframes, diseño visual y prototipado de interfaces interactivas.	Diseño y validación de experiencias de usuario.
Sketch	Software de diseño vectorial.	Orientado al diseño de interfaces digitales, especialmente para aplicaciones web y móviles.	Creación de layouts, componentes y prototipos.
Balsamiq	Herramienta para wireframes de baja fidelidad.	Permite crear bocetos rápidos de interfaces con un enfoque estructural.	Diseño inicial de estructuras de interfaz.
InVision	Plataforma de prototipado y colaboración.	Permite convertir diseños en prototipos interactivos y compartirlos para revisión.	Pruebas de interacción y validación del diseño.

El uso de estas herramientas facilita el proceso de diseño de interfaces, ya que permiten visualizar las ideas, validar la experiencia de usuario y realizar ajustes antes de iniciar el desarrollo del sistema. De esta manera, se optimiza el proceso de programación y se reduce la posibilidad de errores en las etapas posteriores del proyecto.

4. Maquetación de aplicaciones web

La maquetación de aplicaciones web corresponde al proceso mediante el cual se organiza y estructura visualmente el contenido de una aplicación dentro de una página web, siguiendo un diseño previamente definido. A través de este proceso, los diseños conceptuales o gráficos se transforman en código que permite representar la interfaz en el navegador.

Para realizar la maquetación se utilizan principalmente tecnologías como HTML, encargado de definir la estructura del contenido, y CSS, que permite establecer la apariencia visual y la disposición de los elementos dentro de la página.

Desde el desarrollo de software, la maquetación funciona como una etapa intermedia entre el diseño de la interfaz y la programación de la lógica de la aplicación. Aunque no gestiona directamente la funcionalidad del sistema, sí determina cómo se presentan los contenidos, cómo se organizan los componentes y cómo interactúa el usuario con la información.

Una maquetación adecuada contribuye a que la interfaz sea clara, ordenada y coherente, lo que facilita la navegación y mejora la experiencia del usuario. Además, en las aplicaciones web actuales es necesario considerar aspectos como la adaptabilidad a diferentes dispositivos, el rendimiento del sitio y el cumplimiento de criterios de accesibilidad.

4.1. Lenguaje HTML y uso de etiquetas

HTML es un lenguaje de marcado diseñado para describir la estructura de los documentos que se publican en la web. Su función principal es organizar el contenido

de una página, indicando qué partes corresponden a títulos, párrafos, enlaces, imágenes, listas, formularios u otros elementos.

Desde el punto de vista del desarrollo de soluciones de software, HTML actúa como la capa estructural de una aplicación web. Es el primer nivel que el navegador interpreta para presentar la información al usuario. Una estructura HTML bien definida permite que el contenido sea comprendido no solo por las personas, sino también por navegadores, motores de búsqueda y tecnologías de asistencia.

HTML funciona mediante un sistema de etiquetas que rodean el contenido y le asignan un significado específico. Este enfoque permite separar el contenido de su presentación visual y de su comportamiento, facilitando el mantenimiento, la escalabilidad y la interoperabilidad del software.

Características principales:

- HTML significa Lenguaje de Marcado de Hipertexto.
- Es el lenguaje de marcado estándar para crear páginas web.
- Define la estructura de una página web.
- Los elementos HTML indican al navegador cómo mostrar el contenido visualmente.
- Las etiquetas HTML permiten identificar fragmentos de contenido como encabezados, párrafos, enlaces, imágenes u otros componentes de la página.

Aprender mediante ejemplos

Con editores de texto como Bloc de notas en Windows, TextEdit en Mac o editores gratuitos disponibles en internet, es posible crear y editar código HTML y visualizar el resultado en un navegador web.

Ejemplo 1. Estructura básica de una página HTML

A continuación, se presenta un ejemplo sencillo que demuestra la estructura mínima de un documento HTML.

```
<!DOCTYPE html>

<html lang="es">

<head>

  <meta charset="UTF-8">

  <title>Page Title</title>

</head>

<body>

<h1>Aquí es donde va el título del contenido textual</h1>

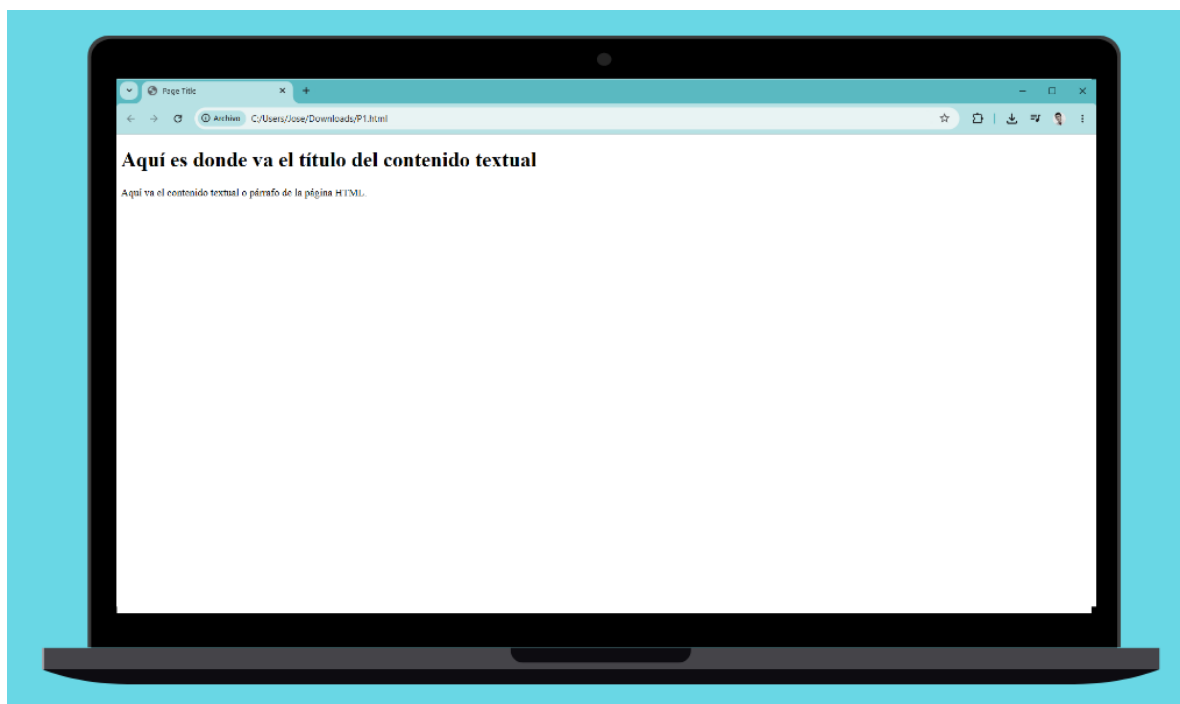
<p>Aquí va el contenido textual o párrafo de la página HTML.</p>

</body>

</html>
```

Nota. El código anterior puede copiarse en un editor de texto y guardarse con la extensión .html. Posteriormente, al abrir el archivo en un navegador web, se visualizará la página generada a partir de ese código.

Figura 5. Ejemplo de primer código HTML



En HTML existen diferentes etiquetas que permiten estructurar y organizar el contenido de una página web. A continuación, se presentan algunas de las etiquetas más utilizadas y su función dentro de una página.

Tabla 6. Tipos de etiquetas HTML

Etiqueta	Descripción	Ejemplo
<h1> a <h6>	Define encabezados o títulos de diferentes niveles de importancia dentro de la página.	Título
<p>	Define un párrafo de texto.	Texto de ejemplo

Etiqueta	Descripción	Ejemplo
<a>	Crea enlaces o hipervínculos que permiten navegar a otras páginas o recursos.	Ir
	Inserta una imagen en la página web.	
	Define una lista no ordenada (con viñetas).	Elemento

Ejemplo 2. Página con múltiples etiquetas HTML

A continuación, se presenta un ejemplo más completo de una página web que utiliza varias de las etiquetas HTML mencionadas anteriormente. Este ejemplo incluye encabezados, párrafos, enlaces de navegación, listas, imágenes, contenedores y un formulario básico.

```
<!DOCTYPE html>
```

```
<html lang="es">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Ejemplo de etiquetas HTML</title>
```

```
</head>
```

```
<body>
```

```
<header>

  <h1>Mi página web</h1>

  <p>Este es un ejemplo de una página con varias etiquetas HTML.</p>

</header>

<nav>

  <a href="#inicio">Inicio</a> |

  <a href="#contenido">Contenido</a> |

  <a href="#contacto">Contacto</a>

</nav>

<section id="inicio">

  <h2>Encabezados en HTML</h2>

  <h1>Encabezado h1</h1>

  <h2>Encabezado h2</h2>

  <h3>Encabezado h3</h3>

  <h4>Encabezado h4</h4>

  <h5>Encabezado h5</h5>

  <h6>Encabezado h6</h6>

</section>

<section id="contenido">
```

`<h2>Párrafos y formato de texto</h2>`

`<p>Este es un párrafo de ejemplo en HTML.</p>`

`<p>Aquí se utiliza <strong>texto en negrita</strong> y <em>texto en cursiva</em>.</p>`

`<h2>Imagen</h2>`

``

`<h2>Lista no ordenada</h2>`

`<ul>`

`<li>Elemento 1</li>`

`<li>Elemento 2</li>`

`<li>Elemento 3</li>`

`</ul>`

`<h2>Lista ordenada</h2>`

`<ol>`

`<li>Paso 1</li>`

`<li>Paso 2</li>`

`<li>Paso 3</li>`

`</ol>`

`<div>`

```
<h3>Contenedor div</h3>
```

```
<p>Este contenido se encuentra dentro de un contenedor  
<strong>div</strong>.</p>
```

```
</div>
```

```
</section>
```

```
<section id="contacto">
```

```
<h2>Formulario de contacto</h2>
```

```
<form>
```

```
<label for="nombre">Nombre:</label>
```

```
<input type="text" id="nombre" name="nombre"><br><br>
```

```
<label for="correo">Correo electrónico:</label>
```

```
<input type="email" id="correo" name="correo"><br><br>
```

```
<button type="submit">Enviar</button>
```

```
</form>
```

```
</section>
```

```
<footer>
```

```
<p>Pie de página - Ejemplo HTML</p>
```

```
</footer>
```

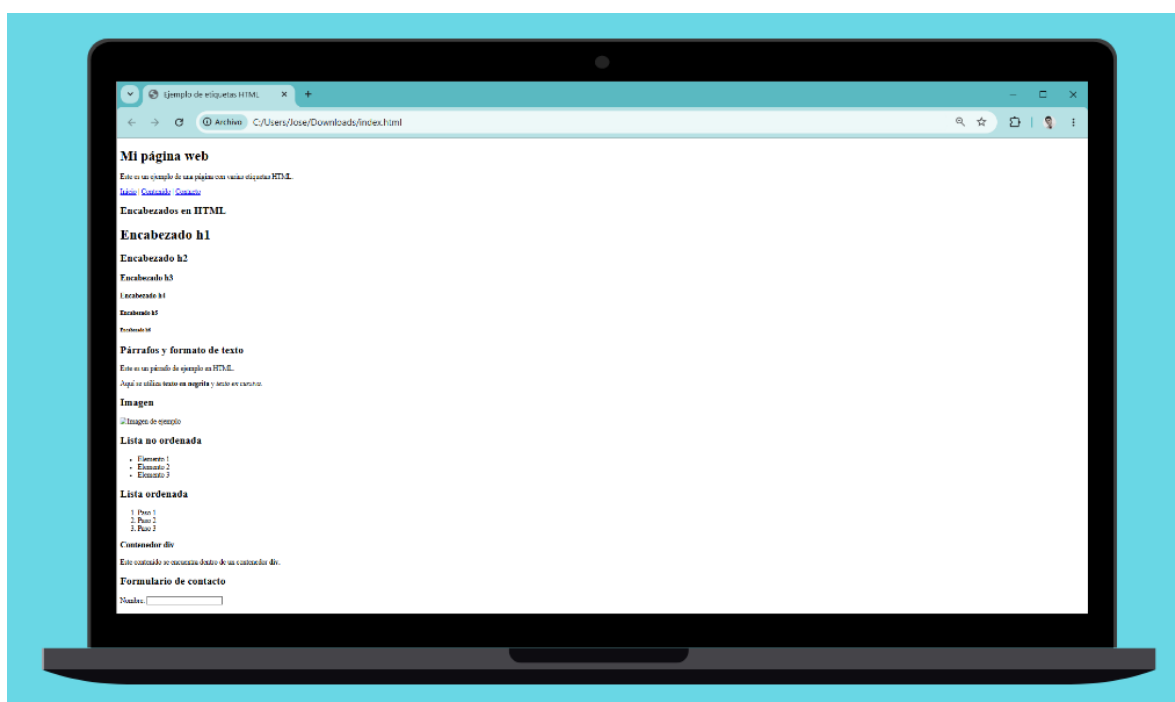
```
</body>
```


</html>

Para usar este ejemplo:

- Abrir un editor de texto, por ejemplo, Visual Studio Code, Bloc de notas u otro editor de código.
- Copiar el código HTML presentado anteriormente.
- Guardar el archivo con el nombre index.html.
- Abrir el archivo guardado en cualquier navegador web para visualizar la página generada.

Figura 6. Página generada a partir del código anterior



Finalmente, al visualizar el resultado en el navegador se observa la estructura de la página con encabezados, párrafos, listas, imágenes y formularios. Sin embargo, el

diseño visual aún es básico y carece de estilos avanzados o elementos dinámicos, los cuales se incorporan mediante hojas de estilo CSS y lenguajes de programación como JavaScript.

4.2. Hojas de estilo CSS

Las hojas de estilo en cascada, conocidas como CSS (Cascading Style Sheets), son un lenguaje de diseño utilizado para describir la presentación de documentos escritos en HTML. Su función principal consiste en definir cómo deben mostrarse los distintos elementos estructurales de una página web, sin modificar el contenido en sí.

Desde la perspectiva del desarrollo de software, CSS permite separar responsabilidades dentro del desarrollo web. Mientras HTML se encarga de la estructura lógica y semántica del contenido, CSS se utiliza para controlar la apariencia visual, como colores, tipografías, tamaños, márgenes y distribución de los elementos en la página. Esta separación facilita el mantenimiento del código, mejora la reutilización de estilos y permite realizar cambios visuales sin alterar la estructura del documento.

El término “en cascada” hace referencia al mecanismo mediante el cual se pueden aplicar múltiples reglas de estilo a un mismo elemento, siguiendo un orden de prioridad. Este sistema permite que los estilos se combinen y sobrescriban según su especificidad y ubicación dentro del documento.

Los estilos CSS pueden incorporarse en una página web de diferentes maneras:

- Atributo style: permite aplicar estilos directamente dentro de una etiqueta HTML, lo que se conoce como estilo en línea.

- Elemento `<style>`: permite definir estilos CSS dentro del mismo documento HTML, generalmente dentro de la sección `<head>`.
- Elemento `<link>`: permite vincular un archivo externo de CSS que contiene los estilos de la página.
- Elemento `<head>`: sección del documento HTML donde normalmente se incluyen los elementos `<style>` y `<link>` para definir o enlazar los estilos.

Entre las propiedades más utilizadas en CSS se encuentran:

- **color**: define el color del texto.
- **font-family**: establece la fuente tipográfica del texto.
- **font-size**: determina el tamaño del texto.
- **border**: define los bordes de un elemento.
- **padding**: controla el espacio interno entre el contenido y el borde del elemento.
- **margin**: controla el espacio externo entre un elemento y otros elementos de la página.

Ejemplo de código HTML con CSS

A continuación, se presenta un ejemplo de una página HTML que incluye estilos CSS definidos dentro del mismo documento mediante la etiqueta `<style>`.

```
<!DOCTYPE html>
```

```
<html lang="es">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Ejemplo de etiquetas HTML con CSS</title>
```

```
<!-- Estilos CSS internos -->
```

```
<style>
```

```
  body {
```

```
    font-family: Arial, sans-serif;
```

```
    margin: 0;
```

```
    background-color: #f4f6f9;
```

```
  }
```

```
  header {
```

```
    background-color: #3498db;
```

```
    color: white;
```

```
    padding: 20px;
```

```
    text-align: center;
```

```
  }
```

```
  nav {
```

```
    background-color: #2c3e50;
```

```
    padding: 10px;
```

```
    text-align: center;
```

```
}  
  
nav a {  
  
    color: white;  
  
    text-decoration: none;  
  
    margin: 0 10px;  
  
    font-weight: bold;  
  
}  
  
nav a:hover {  
  
    color: #f1c40f;  
  
}  
  
section {  
  
    padding: 20px;  
  
    margin: 20px;  
  
    background-color: white;  
  
    border-radius: 8px;  
  
    border: 1px solid #ddd;  
  
}  
  
h1, h2, h3, h4, h5, h6 {  
  
    color: #2c3e50;
```

```
}  
  
p {  
    font-size: 16px;  
    color: #555;  
}  
  
img {  
    display: block;  
    margin: 15px auto;  
    border-radius: 8px;  
}  
  
ul, ol {  
    margin-left: 20px;  
}  
  
li {  
    margin: 5px 0;  
}  
  
div {  
    border: 2px dashed #3498db;  
    padding: 10px;
```

```
margin-top: 10px;

background-color: #ecf0f1;
}

form {

margin-top: 15px;
}

input {

padding: 8px;

margin: 5px 0;

width: 100%;

max-width: 300px;
}

button {

background-color: #3498db;

color: white;

padding: 10px 20px;

border: none;

border-radius: 5px;

cursor: pointer;
```

```
}

button:hover {

    background-color: #2980b9;

}

footer {

    background-color: #2c3e50;

    color: white;

    text-align: center;

    padding: 15px;

    margin-top: 20px;

}

</style>

</head>

<body>

    <!-- Encabezado de la página -->

    <header>

        <h1>Mi página web</h1>

        <p>Ejemplo de estructura HTML con estilos CSS.</p>

    </header>
```



```
<!-- Menú de navegación -->

<nav>

    <a href="#inicio">Inicio</a>

    <a href="#contenido">Contenido</a>

    <a href="#contacto">Contacto</a>

</nav>

<!-- Sección de encabezados -->

<section id="inicio">

    <h2>Encabezados en HTML</h2>

    <h1>Encabezado h1</h1>

    <h2>Encabezado h2</h2>

    <h3>Encabezado h3</h3>

    <h4>Encabezado h4</h4>

    <h5>Encabezado h5</h5>

    <h6>Encabezado h6</h6>

</section>

<!-- Contenido principal -->

<section id="contenido">

    <h2>Párrafos y formato de texto</h2>
```

`<p>Este es un párrafo de ejemplo en HTML.</p>`

`<p>`

`Aquí se muestra <strong>texto en negrita</strong> y`

`<em>texto en cursiva</em>.`

`</p>`

`<h2>Imagen</h2>`

``

`<h2>Lista no ordenada</h2>`

`<ul>`

`<li>Elemento 1</li>`

`<li>Elemento 2</li>`

`<li>Elemento 3</li>`

`</ul>`

`<h2>Lista ordenada</h2>`

`<ol>`

`<li>Paso 1</li>`

`<li>Paso 2</li>`

```
<li>Paso 3</li>
```

```
</ol>
```

```
<div>
```

```
<h3>Contenedor div</h3>
```

```
<p>
```

Este contenido está dentro de un contenedor

`<strong>div</strong>`, el cual se utiliza para agrupar
elementos dentro de una página web.

```
</p>
```

```
</div>
```

```
</section>
```

```
<!-- Formulario -->
```

```
<section id="contacto">
```

```
<h2>Formulario de contacto</h2>
```

```
<form action="#" method="post">
```

```
<label for="nombre">Nombre</label><br>
```

```
<input type="text" id="nombre" name="nombre"><br><br>
```

```
<label for="correo">Correo electrónico</label><br>
```

```
<input type="email" id="correo" name="correo"><br><br>
```

```

        <button type="submit">Enviar</button>

    </form>

</section>

<!-- Pie de página -->

<footer>

    <p>Pie de página – Ejemplo de HTML y CSS</p>

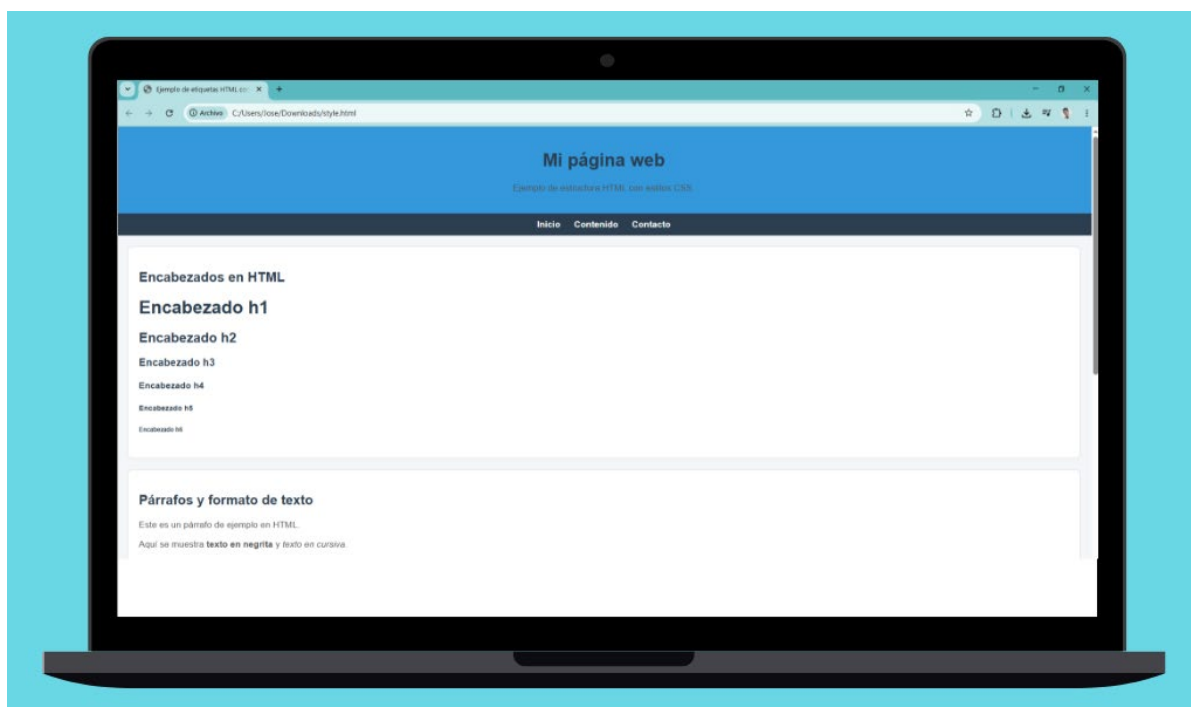
</footer>

</body>

</html>

```

Figura 7. HTML con estilos CSS



El desarrollo de este código HTML con integración de estilos CSS permite comprender de manera práctica cómo se construye y presenta una página web básica. A través del uso de etiquetas estructurales como `<header>`, `<nav>`, `<section>` y `<footer>`, se organiza el contenido de forma lógica y semántica, lo que facilita tanto la lectura del código como la navegación del usuario.

La incorporación de CSS mediante la etiqueta `<style>` permite mejorar la apariencia visual del sitio web mediante propiedades como color, tipografía, márgenes, bordes y espaciados. Esto contribuye a crear interfaces más atractivas, organizadas y fáciles de usar.

Asimismo, el uso de elementos como listas, imágenes y formularios muestra cómo HTML permite estructurar diferentes tipos de contenido, mientras que CSS optimiza su presentación visual. La combinación de ambas tecnologías constituye la base del desarrollo frontend y resulta fundamental para la creación de aplicaciones web modernas.

Para complementar la información, acceder al video: Aplicando el CSS.

https://www.youtube.com/watch?v= qlj_FyUR-w

4.3. Estilos y guías de estilo para la presentación web

Los estilos en la web se refieren al conjunto de características visuales que determinan la apariencia de una página o aplicación. Estos estilos son implementados principalmente mediante CSS (Cascading Style Sheets), el cual permite definir colores, tipografías, tamaños, espacios, alineaciones y otros aspectos visuales de la interfaz.

Los estilos permiten transformar una estructura HTML básica en una interfaz visualmente organizada, atractiva y funcional. Además, facilitan la adaptación del contenido a diferentes dispositivos mediante técnicas como el diseño responsivo, lo cual mejora la accesibilidad y la experiencia de usuario.

Los estilos presentan diversas características que los convierten en un elemento fundamental dentro del desarrollo web:

- Permiten separar el contenido del diseño.
- Mejoran la apariencia visual de la interfaz.
- Facilitan la reutilización de componentes de diseño.
- Permiten mantener una consistencia visual en toda la aplicación.
- Favorecen la adaptación del contenido a diferentes dispositivos y tamaños de pantalla.

Existen diferentes formas de aplicar estilos en una página web, dependiendo de las necesidades del proyecto y de la organización del código.

- a. Estilos en línea (Inline CSS):** se aplican directamente dentro de una etiqueta HTML utilizando el atributo `style`. Generalmente se utilizan para realizar pruebas rápidas o aplicar modificaciones puntuales. Sin embargo, no se recomiendan en proyectos grandes debido a que dificultan el mantenimiento del código y no permiten reutilizar estilos.

Ejemplo en HTML:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1 style="color: blue;">Título en azul</h1>
```

```
<p style="font-size: 18px;">Este es un párrafo con tamaño de letra  
personalizado.</p>
```

```
</body>
```

```
</html>
```

- b. Estilos internos (Internal CSS):** se definen dentro del mismo archivo HTML, utilizando la etiqueta `style` ubicada en la sección `head` del documento. Esta forma se utiliza cuando los estilos solo serán aplicados a una página específica.

Ejemplo en HTML:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
h1 {
```

```
color: red;
```

```
}
```

```
p {
```

```
font-size: 18px;
```

```
    color: gray;

}

</style>

</head>

<body>

<h1>Título en rojo</h1>

<p>Este es un párrafo con estilos internos.</p>

</body>

</html>
```

- c. Estilos externos (External CSS):** se escriben en un archivo independiente con extensión .css y posteriormente se enlazan al documento HTML mediante la etiqueta link. Esta es la forma más recomendada en el desarrollo profesional, ya que permite reutilizar estilos en múltiples páginas y facilita el mantenimiento del código.

Ejemplo en HTML:

```
<!DOCTYPE html>

<html>

<head>

<link rel="stylesheet" href="estilos.css">

</head>
```



```
<body>

<h1>Título con CSS externo</h1>

<p>Este es un párrafo con estilos externos.</p>

</body>

</html>
```

Con el fin de sintetizar las diferentes formas de aplicar estilos, a continuación, se presenta una comparación entre las principales formas de aplicar CSS en una página web.

Tabla 7. Comparación de formas de aplicar estilos CSS

Tipo de estilo	Descripción	Ventajas	Limitaciones
Estilos en línea (Inline CSS)	Los estilos se aplican directamente dentro de una etiqueta HTML mediante el atributo style.	Permiten realizar cambios rápidos en elementos específicos.	No permiten reutilización de estilos y dificultan el mantenimiento del código en proyectos grandes.
Estilos internos (Internal CSS)	Los estilos se definen dentro del mismo documento HTML mediante la etiqueta <style> ubicada en la sección <head>.	Permiten aplicar estilos a una página completa sin crear archivos adicionales.	Los estilos solo funcionan dentro de esa página y no se pueden reutilizar fácilmente en otros documentos.
Estilos externos (External CSS)	Los estilos se escriben en un archivo independiente con extensión .css y se enlazan al documento HTML mediante la etiqueta <link>.	Facilitan la reutilización de estilos, mejoran la organización del código y permiten mantener consistencia en múltiples páginas.	Requieren crear y gestionar archivos adicionales dentro del proyecto.

Las guías de estilo son documentos o conjuntos de reglas que establecen cómo deben aplicarse los elementos visuales dentro de una página o aplicación web. Estas guías permiten mantener uniformidad en aspectos como el uso de colores, tipografías, tamaños de texto, espacios, botones, iconos y componentes de interfaz.

La implementación de guías de estilo facilita el trabajo colaborativo entre diseñadores y desarrolladores, además de garantizar que todas las páginas de una aplicación mantengan una apariencia coherente y profesional.

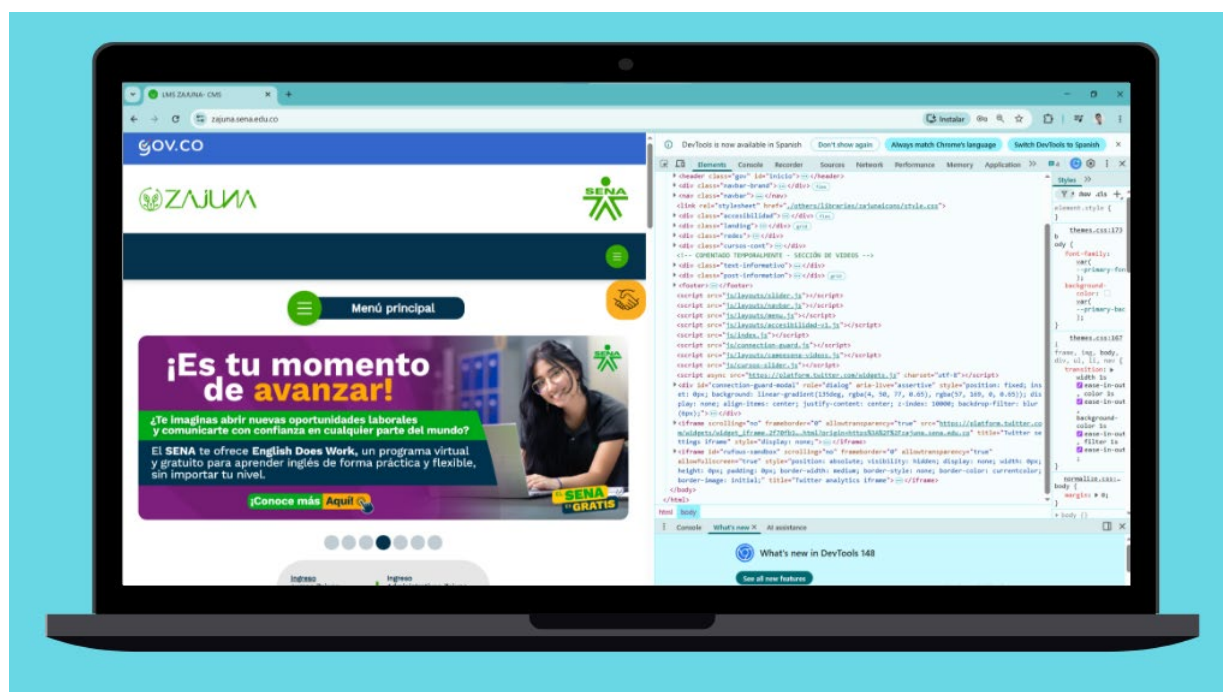
Los estilos y las guías de estilo son elementos fundamentales en la presentación web. Su correcta aplicación permite crear interfaces claras, atractivas y funcionales, mejorando la experiencia del usuario y facilitando el desarrollo y mantenimiento de aplicaciones web.

5. Tecnologías frontend para el desarrollo web

El término frontend hace referencia al conjunto de tecnologías, herramientas y técnicas utilizadas para desarrollar la parte visual e interactiva de una aplicación web. Esta capa corresponde a todo aquello con lo que el usuario interactúa directamente dentro de una página o sistema web, y se ejecuta en el navegador del usuario.

Desde el punto de vista del desarrollo de soluciones de software, el frontend se encarga de presentar la información que genera el backend y de capturar las acciones realizadas por el usuario, tales como clics, desplazamientos, ingreso de datos o selección de opciones. Estas acciones son posteriormente procesadas por otras capas del sistema para generar respuestas y actualizar la información mostrada en la interfaz.

Figura 8. Ejemplo de frontend y backend en Zajuna



Nota. Tomado de Zajuna (s. f.).

En el entorno de aprendizaje Zajuna se puede presentar esta diferencia con claridad. La parte visible para el usuario corresponde al frontend, donde se presentan los contenidos, menús, botones y formularios. Por su parte, el backend corresponde al conjunto de códigos y procesos que funcionan en segundo plano y que permiten gestionar la información del sistema, incluyendo estructuras desarrolladas mediante HTML, CSS y JavaScript.

Dentro del desarrollo frontend intervienen diversas tecnologías que permiten construir la estructura, el diseño y la interactividad de las páginas web. Entre las principales se encuentran:

- **HTML (HyperText Markup Language):** es el lenguaje de marcado que define la estructura del contenido en la web. Permite organizar textos, imágenes, enlaces, formularios y otros elementos que componen una página web.
- **CSS (Cascading Style Sheets):** se encarga de la presentación visual del contenido. A través de este lenguaje se pueden definir colores, tipografías, tamaños, márgenes, alineaciones y otros aspectos visuales que mejoran la apariencia de la interfaz y permiten adaptar el diseño a diferentes dispositivos.
- **JavaScript:** es un lenguaje de programación que permite añadir interactividad a las páginas web. Gracias a este lenguaje es posible validar formularios, modificar elementos del documento, generar animaciones y crear aplicaciones dinámicas. Elementos como movimientos de interfaz, resaltado de contenido o validaciones de datos en plataformas educativas son ejemplos de funcionalidades implementadas con JavaScript.

- **Frameworks y bibliotecas frontend:** en el desarrollo web moderno, las tecnologías frontend se complementan con el uso de frameworks y bibliotecas que facilitan la creación de interfaces complejas. Estas herramientas proporcionan estructuras, componentes reutilizables y funciones predefinidas que agilizan el desarrollo, mejoran la organización del código y permiten construir aplicaciones web más eficientes y escalables.

5.1. Funcionalidades de la tecnología frontend

La tecnología frontend hace referencia al conjunto de herramientas, lenguajes y técnicas utilizados para desarrollar la parte visual e interactiva de una aplicación web. Esta capa del software se ejecuta directamente en el navegador del usuario y permite mostrar información, recibir datos, responder a las acciones del usuario y facilitar la interacción con el sistema.

Desde la perspectiva del desarrollo de soluciones de software, el frontend actúa como intermediario entre el usuario y el backend, que es la capa encargada de procesar la información, ejecutar la lógica de negocio y gestionar las bases de datos. Mientras el backend se ocupa del procesamiento interno del sistema, el frontend presenta esa información de forma clara, organizada y accesible para el usuario.

El frontend no se limita únicamente al diseño visual de las páginas web. También incluye aspectos estructurales y funcionales que determinan cómo se organiza el contenido, cómo se comporta la interfaz y cómo se guía al usuario durante la navegación dentro de la aplicación.

Video 1. Funcionalidades de la tecnología frontend



Enlace de reproducción del video

Transcripción del video. Funcionalidades de la tecnología frontend

El frontend corresponde a la capa del software que se ejecuta directamente en el navegador del usuario y que permite visualizar la información, interactuar con el sistema y facilitar la comunicación entre la persona y la aplicación.

Desde el desarrollo de soluciones de software, el frontend cumple un papel fundamental, ya que transforma los datos y procesos del sistema en interfaces comprensibles, organizadas e intuitivas. A través de esta capa, el usuario puede navegar por páginas, completar formularios, consultar información y ejecutar diferentes acciones dentro de una aplicación web.

El funcionamiento del frontend puede entenderse como un proceso de interacción entre el usuario y el sistema:

Transcripción del video. Funcionalidades de la tecnología frontend

Usuario → Interfaz web → Navegador → Comunicación con el backend → Respuesta del sistema → Actualización de la interfaz.

Para cumplir estas funciones, el desarrollo frontend se apoya principalmente en tres tecnologías fundamentales:

HTML, encargado de estructurar el contenido de la página web, como textos, imágenes, enlaces y formularios.

CSS, responsable de la presentación visual, permitiendo definir colores, tipografías, tamaños, espacios y distribución de los elementos.

y

JavaScript, que permite agregar interactividad a las páginas web mediante validación de datos, manipulación de elementos y actualización dinámica del contenido.

Estas tecnologías trabajan de manera conjunta para construir interfaces que permitan una interacción eficiente entre el usuario y el sistema. Comprender las funcionalidades del frontend permite desarrollar aplicaciones web más accesibles, dinámicas y orientadas a mejorar la experiencia del usuario.

Entre las principales características de la tecnología frontend se destacan:

- Interacción directa con el usuario.
- Ejecución en el navegador web.

- Integración con servicios del backend.
- Enfoque en la experiencia de usuario.
- Uso de tecnologías estándar de la web.

Las funcionalidades del frontend permiten que una aplicación web sea dinámica, interactiva y fácil de utilizar. Entre las más importantes se encuentran:

- **Presentación de contenido:** permite presentar información de manera organizada y visualmente clara mediante el uso de tecnologías como HTML y CSS.
- **Interacción con el usuario:** gestiona eventos como clics, desplazamientos, envíos de formularios o selección de opciones dentro de la interfaz.
- **Validación de datos:** permite verificar la información ingresada por el usuario antes de enviarla al servidor, ayudando a prevenir errores y mejorar la experiencia de uso.
- **Consumo de servicios:** facilita la comunicación con el backend mediante servicios o APIs para enviar o recibir información.
- **Actualización dinámica de la interfaz:** permite modificar partes de la página sin necesidad de recargarla completamente, lo que mejora la fluidez de la navegación.

La tecnología frontend constituye un componente fundamental en el desarrollo de soluciones web, ya que permite construir interfaces que conectan al usuario con el sistema de forma clara, eficiente e intuitiva. Su correcta implementación contribuye a mejorar la usabilidad, la accesibilidad y la experiencia general de las aplicaciones web.

5.2. Tipos de datos y funciones en el desarrollo frontend

Los tipos de datos corresponden a una clasificación que define la naturaleza de la información que puede almacenarse, procesarse y manipularse dentro de un programa. Cada tipo de dato establece un conjunto de valores posibles y un conjunto de operaciones que pueden realizarse sobre dichos valores.

En términos simples, los tipos de datos indican al programa qué tipo de información se está manejando y cómo debe interpretarla. Gracias a esta clasificación, los lenguajes de programación pueden procesar correctamente números, textos, valores lógicos u otras estructuras de información.

Desde el punto de vista del desarrollo de soluciones de software, los tipos de datos permiten organizar la información de manera estructurada, reducir ambigüedades y prevenir errores durante la ejecución del sistema. Cuando un programador define un tipo de dato, establece reglas que permiten al lenguaje interpretar los valores correctamente y detectar posibles fallos en el código.

En el desarrollo frontend, especialmente con JavaScript, los tipos de datos permiten gestionar la información que se muestra en la interfaz, validar datos ingresados por el usuario y procesar la información recibida desde servicios o APIs.

La correcta definición y uso de los tipos de datos resulta fundamental para garantizar la calidad y el correcto funcionamiento de una solución de software. Un uso inadecuado puede provocar errores lógicos, fallos de ejecución o comportamientos inesperados del sistema.

Además, los tipos de datos influyen en aspectos como:

- La organización de la información dentro del sistema.

- La eficiencia en el procesamiento de datos.
- La claridad y mantenimiento del código.
- La detección temprana de errores.
- La estabilidad y seguridad del software.

Comprender los tipos de datos constituye uno de los primeros pasos en el aprendizaje de la programación, ya que permite entender cómo los programas representan la información del mundo real mediante estructuras digitales.

Los tipos de datos pueden clasificarse en diferentes categorías según la forma en que almacenan la información.

- Datos primitivos:** representan los tipos básicos que se encuentran en la mayoría de los lenguajes de programación. Se utilizan para almacenar valores simples.

Tabla 8. Datos primitivos

Tipo de dato	Descripción	Ejemplo
Entero (Integer).	Representa números enteros sin decimales.	10, -5, 0
Decimal (Float or Double).	Representa números con decimales.	3.14, 0.5
Cadena de texto (String).	Representa secuencias de caracteres o texto.	"Hola mundo"
Booleano (Boolean).	Representa valores lógicos.	true, false

- b. Datos compuestos o estructuras de datos:** los datos compuestos permiten almacenar múltiples valores dentro de una misma estructura, lo que facilita la organización de información más compleja.

Tabla 9. Datos compuestos

Tipo	Descripción	Ejemplo
Arreglos (Arrays).	Permiten almacenar una colección de elementos del mismo tipo.	[1, 2, 3, 4]
Objetos (Objects).	Representan entidades mediante pares clave-valor.	{nombre: "Ana", edad: 25}
Listas.	Estructuras dinámicas que pueden crecer o reducirse.	["rojo", "verde", "azul"]

JavaScript es uno de los lenguajes principales del desarrollo frontend y utiliza varios tipos de datos para gestionar la información dentro de las aplicaciones web.

- **Number:** representa valores numéricos enteros o decimales.
- **String:** representa texto o cadenas de caracteres.
- **Boolean:** representa valores lógicos verdadero o falso.
- **Object:** representa estructuras de datos complejas.
- **Array:** permite almacenar listas de elementos.
- **Null:** representa la ausencia intencional de valor.
- **Undefined:** indica que una variable no tiene valor asignado.

En el desarrollo frontend, las aplicaciones web frecuentemente necesitan comunicarse con servidores o servicios externos para obtener o enviar información. Este proceso se conoce como consumo de servicios o consumo de APIs.

Las funciones de consumo permiten que una aplicación web solicite datos a un servidor, envíe información o actualice registros dentro de un sistema. Estas operaciones se realizan generalmente mediante protocolos web como HTTP.

Tabla 10. Métodos HTTP utilizados en el consumo de servicios

Método	Función	Ejemplo de uso
GET	Obtener información desde un servidor.	Consultar una lista de usuarios.
POST	Enviar nuevos datos al servidor.	Registrar un nuevo usuario.
PUT	Actualizar información existente.	Modificar los datos de un usuario.
DELETE	Eliminar información.	Eliminar un registro de la base de datos.

Estas funciones de consumo permiten que las aplicaciones web interactúen con bases de datos, sistemas externos o servicios en la nube, lo que hace posible desarrollar aplicaciones dinámicas y conectadas.

En el desarrollo frontend moderno, este proceso suele realizarse mediante herramientas o funciones como `fetch` o bibliotecas de comunicación con APIs.

5.3. Uso de librerías en el desarrollo frontend

En programación, una librería es un conjunto de funciones, clases, componentes o recursos reutilizables diseñados para resolver problemas comunes de forma estandarizada. En el desarrollo frontend, las librerías se utilizan para facilitar tareas relacionadas con la construcción de interfaces de usuario, la manipulación del DOM, la

gestión de datos, la comunicación con servicios externos y la implementación de interacciones dinámicas.

Desde el punto de vista del desarrollo de soluciones de software, una librería frontend actúa como un recurso que amplía las capacidades del desarrollador. En lugar de escribir código desde cero para cada funcionalidad, se pueden reutilizar soluciones previamente desarrolladas y probadas, lo que permite reducir el tiempo de desarrollo, mejorar la calidad del código y disminuir la probabilidad de errores.

Es importante diferenciar las librerías de otros elementos como los frameworks. Mientras una librería se enfoca en resolver una tarea específica y puede integrarse de forma flexible dentro de un proyecto, un framework establece una estructura más completa para el desarrollo de la aplicación. Comprender esta diferencia permite seleccionar adecuadamente las herramientas según las necesidades del proyecto.

En el desarrollo frontend moderno existe una amplia variedad de librerías que facilitan diferentes tareas relacionadas con la interfaz, la gestión de datos, las animaciones o la comunicación con servicios externos.

Tabla 11. Ejemplos de librerías utilizadas en el desarrollo frontend

Librería	Uso principal	Características
React	Construcción de interfaces de usuario.	Basado en componentes y uso de Virtual DOM.
jQuery	Manipulación del DOM.	Simplifica la interacción con HTML, eventos y animaciones.
Axios	Consumo de APIs.	Permite realizar peticiones HTTP de forma sencilla.

Librería	Uso principal	Características
Lodash	Manipulación de datos.	Conjunto de funciones utilitarias para trabajar con arreglos y objetos.
Chart.js	Visualización de datos.	Permite crear gráficos interactivos.
D3.js	Visualización avanzada de datos.	Generación de gráficos dinámicos y personalizados.
Three.js	Gráficos 3D.	Renderizado de objetos 3D en el navegador.
Anime.js	Animaciones.	Animaciones ligeras y fluidas.
GSAP	Animaciones avanzadas.	Alto rendimiento en animaciones complejas.
Swiper.js	Carruseles.	Creación de sliders y carruseles responsivos.
Moment.js	Manejo de fechas.	Manipulación y formato de fechas.
Day.js	Manejo de fechas.	Alternativa ligera a Moment.js.
Alpine.js	Interactividad.	Framework ligero para agregar comportamiento a HTML.
Redux	Gestión de estado.	Control centralizado del estado de la aplicación.
Zustand	Gestión de estado.	Librería ligera para manejar estados globales.
Socket.io	Comunicación en tiempo real.	Permite interacción bidireccional entre cliente y servidor.
RxJS	Programación reactiva.	Manejo de eventos y datos asíncronos.
Bootstrap JS	Componentes interactivos.	Funcionalidades dinámicas para componentes de interfaz.
SweetAlert2	Alertas visuales.	Ventanas emergentes personalizadas.

Librería	Uso principal	Características
Formik	Gestión de formularios.	Manejo del estado y validación de formularios.
Yup	Validación de datos.	Validación estructurada de formularios.
FullCalendar	Calendarios interactivos.	Gestión visual de eventos.
p5.js	Gráficos y arte visual.	Creación de visualizaciones y animaciones artísticas.

Ejemplo de uso de la librería Axios para consumir una API

A continuación, se presenta un ejemplo sencillo de cómo utilizar la librería Axios para obtener información desde un servicio web. En este caso, el código realiza una solicitud HTTP de tipo GET a una API pública y presenta los datos obtenidos en la consola del navegador.

```

<!DOCTYPE html>

<html>

<head>

<title>Ejemplo Axios</title>

<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>

</head>

<body>

<h2>Consumo de API con Axios</h2>

```

```
<button onclick="obtenerDatos()">Obtener datos</button>

<script>

function obtenerDatos() {

  axios.get('https://jsonplaceholder.typicode.com/users')

    .then(function(response) {

      console.log(response.data);

      alert("Datos obtenidos correctamente. Revisa la consola.");

    })

    .catch(function(error) {

      console.error("Error en la petición:", error);

    });

}

</script>

</body>

</html>
```

En este ejemplo, la librería Axios permite realizar una petición a una API externa de forma sencilla. Cuando el usuario presiona el botón Obtener datos, el navegador envía una solicitud al servicio web y recibe una respuesta en formato JSON. Posteriormente, los datos pueden procesarse y presentarse en la interfaz de la aplicación.

Este tipo de herramientas facilita la comunicación entre el frontend y el backend, permitiendo que las aplicaciones web obtengan información de servidores externos de manera rápida y eficiente.

El uso de librerías en el desarrollo frontend constituye una práctica fundamental en la programación de soluciones de software modernas. Estas herramientas permiten reutilizar código, mejorar la productividad, mantener estándares de desarrollo y construir interfaces más robustas y funcionales.

Desde una perspectiva educativa y profesional, comprender cómo seleccionar e integrar librerías frontend es una competencia clave para los desarrolladores. Su uso adecuado contribuye a la construcción de aplicaciones web eficientes, mantenibles y alineadas con las necesidades de los usuarios y los estándares actuales del desarrollo web.

Síntesis

El componente aborda los fundamentos del diseño y desarrollo de aplicaciones web desde la organización de la información hasta la implementación de tecnologías frontend. Inicialmente se estudia la arquitectura de la información, enfocándose en la organización del contenido y los sistemas de navegación que facilitan la interacción del usuario con el sitio web. Posteriormente se analizan los principios del diseño de contenido e información web, incluyendo la distribución del contenido, los formatos de archivos para la publicación de recursos y herramientas de planificación visual como el wireframe, la jerarquía de información y las retículas. Asimismo, se profundiza en el diseño de interfaces web, considerando los diferentes tipos de interfaces, los principios de diseño intuitivo y usable, y el uso de mockups, prototipos y herramientas especializadas para su creación. Finalmente, se abordan los aspectos relacionados con la maquetación de aplicaciones web mediante HTML y CSS, así como el uso de estilos y guías de estilo, para concluir con una introducción a las tecnologías frontend, sus funcionalidades, el manejo de tipos de datos y el uso de librerías que facilitan el desarrollo de interfaces dinámicas e interactivas.



Glosario

API (Interfaz de Programación de Aplicaciones): conjunto de reglas y herramientas que permiten la comunicación entre diferentes aplicaciones o sistemas para intercambiar información y funcionalidades.

Aplicación web: software que se ejecuta en un navegador y permite a los usuarios interactuar con servicios o contenidos a través de Internet.

Arquitectura de la información: organización estructurada de contenidos y funcionalidades dentro de una aplicación o sitio web para facilitar su comprensión, navegación y uso.

Backend: parte de una aplicación que se ejecuta en el servidor y se encarga del procesamiento de datos, la lógica del sistema y la comunicación con bases de datos.

CSS (Cascading Style Sheets): lenguaje utilizado para definir la apariencia visual de los elementos de una página web, como colores, tipografías, tamaños y distribución.

Frontend: parte visible de una aplicación web con la que interactúa el usuario, desarrollada mediante tecnologías como HTML, CSS y JavaScript.

Framework: conjunto de herramientas, bibliotecas y estructuras que facilitan el desarrollo de aplicaciones al proporcionar una base organizada para la programación.

HTML (HyperText Markup Language): lenguaje de marcado utilizado para estructurar y organizar el contenido de las páginas web.

Interfaz de usuario (UI): conjunto de elementos visuales mediante los cuales una persona interactúa con un sistema o aplicación, como botones, menús, formularios e íconos.

JavaScript: lenguaje de programación que permite agregar interactividad, dinamismo y funcionalidades avanzadas a las páginas web.

Librería: conjunto de funciones o componentes reutilizables que facilitan la implementación de tareas específicas en el desarrollo de software.

Prototipo: representación preliminar de una aplicación o interfaz que permite visualizar y evaluar su estructura, funcionamiento y experiencia de usuario antes del desarrollo final.

Responsive design (Diseño responsivo): enfoque de diseño que permite que una interfaz web se adapte automáticamente a diferentes tamaños de pantalla, como computadores, tabletas y dispositivos móviles.

UX (experiencia de usuario): conjunto de percepciones y sensaciones que experimenta una persona al interactuar con un sistema, producto o servicio digital.

Wireframe: representación visual esquemática de una interfaz que muestra la distribución de los elementos y la estructura básica de una página o aplicación.

Referencias bibliográficas

Córcoles Tendero, J. E., & Montero Simarro, F. (2014). Diseño de interfaces web. RA-MA Editorial.

Ecosistema de Recursos Educativos Digitales SENA. (2023). Aplicando el CSS [Video]. YouTube. https://www.youtube.com/watch?v=qlj_FyUR-w

Ecosistema de Recursos Educativos Digitales SENA. (2023). ¿Cómo realizar un wireframe? [Video]. YouTube. <https://www.youtube.com/watch?v=94UTvXQgRfM>

Gutiérrez, C. (2008). Diseño web y arquitectura de información para sitios 2.0. Cuadernos.Info, (22), 44–51. <https://doi.org/10.7764/cdi.22.90>

Pérez-Montoro, M. (2010). Arquitectura de la información en entornos web. Ediciones Trea.

Powell, T. A. (2000). Web design: The complete reference. Osborne/McGraw-Hill.

Rosenfeld, L., Morville, P., & Arango, J. (2024). Arquitectura de la información (4.ª ed.). O'Reilly Media.

Romero, E. (s. f.). ¿Qué es la retícula en diseño gráfico y para qué sirve? Inesem Business School. <https://www.inesem.es/revistadigital/disenio-y-artes-graficas/que-es-la-reticula-en-diseno-grafico>

Tidwell, J., Brewer, C., & Valencia, A. (2024). Diseño de interfaces (3.ª ed.). O'Reilly Media.

Untitled. (s. f.). CodePen. <https://codepen.io/pen/>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico – Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Jorge Eduardo Rueda Peña	Experto temático	Centro de Comercio y Servicios - Regional Tolima
Viviana Esperanza Herrera Quiñonez	Evaluada instruccional	Centro de Comercio y Servicios - Regional Tolima
Jose Yobani Penagos Mora	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Sebastian Trujillo Afanador	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Gilberto Junior Rodríguez Rodríguez	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
María Fernanda Pineda Mora	Evaluada de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima